

4EYES Labs Audit

Lithos: Mining (Part 2)

Auditor: Luca D'Angelo

Date: TBD

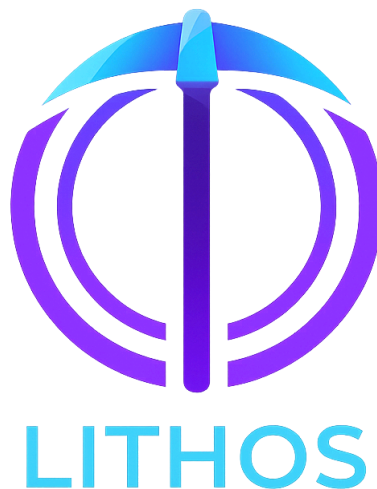


Table of Contents

Table of Contents	2
Disclaimer	3
Audit Summary	4
Contracts	5
Holding	6
Evaluation	10
Payout	11
Provision Guard	12
Provision	13
Liquidity Pool	14
Fee Vault	15
Appendix	16
Issue Guide	17
About Us	17

Disclaimer

This audit is provided as-is at the time of its completion and might not reflect the most up-to-date state of any smart-contracts deployed, or any design document, or any research paper referenced herein.

Audit Summary

Protocol	Lithos: Mining (Part 2)
Repository	https://github.com/Lithos-Protocol/Lithos-Client
Commit	15dfa32a4667eaa71e6bcbbf80a8ed7f1b69890e

Contract	File Name	Severity	Status
Holding	Holding.ergo	WARNING	RESOLVED
Evaluation	Evaluation.ergo	WARNING	RESOLVED
Payout	Payout.ergo	SERIOUS	RESOLVED
Liquidity Pool	LD_LiquidityPool.ergo	WARNING	RESOLVED
Fee Vault	LD_FeeVault.ergo	GOOD	RESOLVED
Provision	LD_Provision.ergo	SERIOUS	RESOLVED
Provision Guard	LD_ProvisionGuard	GOOD	RESOLVED

Contracts

Holding

Description

- This contract holds the ERG block reward provided by the lender's collateral, including the LIT tokens to be emitted to the pool.
- During the submission period, miners may provide their NISP to be included in the NISP tree inside this contract.
- After the submission period ends, no more NISPs can be provided by miners and this contract becomes the evaluation contract where fraud proofs will then be checked.

Box Contents

Registers

- R4 [AvlTree]: The NISP tree, initially an empty AVL tree mapping a miner's hashed proposition bytes to their corresponding NISP.
- R5 [Int]: Total miners in a given period, representing miners who have posted a NISP.
- R6 [BigInt]: The total share score, which increases for every NISP posted. The Share Score of a NISP is the first 8 bytes of the NISP. Individual Share Scores have type Long.
- R7 [Long]: Period start represented by the creation height of the holding box.

Tokens

- Index 0: LIT tokens to be emitted to miners of the pool.

Context Variables

- Index 0 [Coll[Byte]]: CTX_SIGNER is the value bytes of the signer of the transaction used to authenticate submission of the NISPs.
- Index 1 [(Coll[Byte], Coll[Byte])]: CTX_NIS_DATA is the data to be inserted into the NISP tree, a pair of the hashed miner proposition bytes and the corresponding NISP.
- Index 2 [Coll[Byte]]: CTX_NISP_PROOF is the AVL Tree insertion proof.

Compile-Time Constants

- CONST_PERIOD_LENGTH [Int]: The length of the NISP submission period, used for determining the appropriate spending path in the holding contract.
- CONST_EVAL_PROPBYTES [Coll[Byte]]: The hashed proposition bytes of the evaluation contract.

Contract Analysis

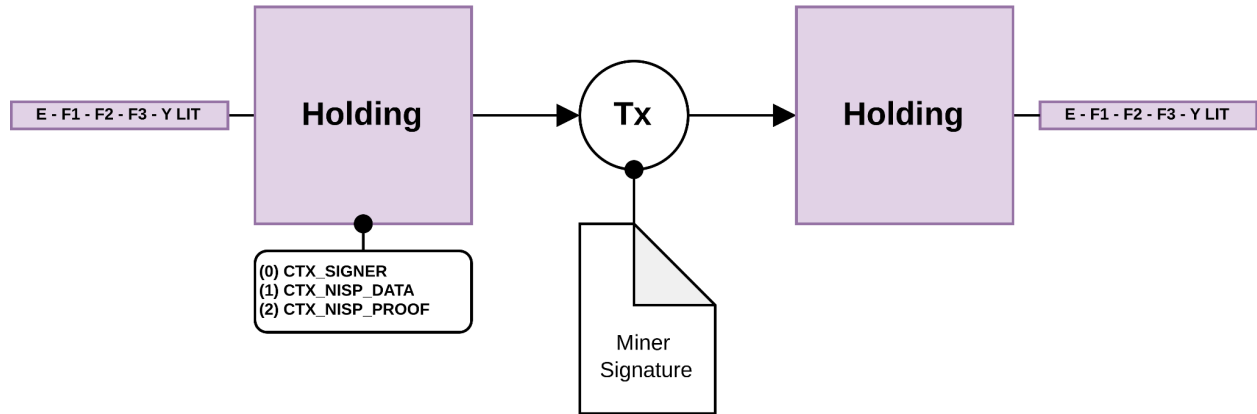
- L54 checks that if the height of the current block being validated is less than the current period start + the period length compile-time constant, thus the holding box is in the NISP submission phase, then the following is checked:
 - L55 defines signerProp, which executes the CTX_SIGNER Coll[Byte] from the transaction context at index 0.
 - CTX_SIGNER is never used anywhere in the contract.
 - How do we know this will be a miner's p2pk and not some other p2s?
 - L56 defines authenticMiner checking that the blake2b256 hash of the proposition bytes of the signerProp SigmaProp equals the key of the provided CTX_NISP_DATA.
 - L57 defines validUTXO checking that the output holding contract proposition bytes matches the current holding contract proposition bytes.
 - L58 defines rewardConserved checking that the ERG value of the output is preserved.
 - L63 defines validNISP checking that:
 - If the provided NISP data in CTX_NISP_DATA has size greater than 8 bytes, then:
 - L65 defines score as the Long obtained from the first 8 bytes of the NISP.
 - The following must all be true:
 - The score must be greater than 0.
 - The total score in the output holding box must be the current total score plus the obtained score converted to BigInt.
 - The NISP size must be greater than or equal the CONST_NISP_MIN value.
 - The NISP size must be strictly less than the CONST_NISP_MAX value.
 - Remaining NISP conditions are checked by the evaluation contract.
 - Otherwise:
 - Return false.
 - L74 defines treeUpdated checking that the current tree with the inserted NISP data equals the tree in the output holding box.
 - A valid insertion means there can be no duplicates, thus no duplicate miner NISP submissions with the same address is possible.
 - L78 defines onlyOne ensuring that the current box id matches the id of the first input box so there are no possible duplicates.
 - L79 defines incrementMiners checking that the output holding box increments the miner NISP submission count.
 - L80 defines samePeriod checking that the output holding box maintains the same period start.

4EYES Labs Audit - Lithos: Mining (Part 2)

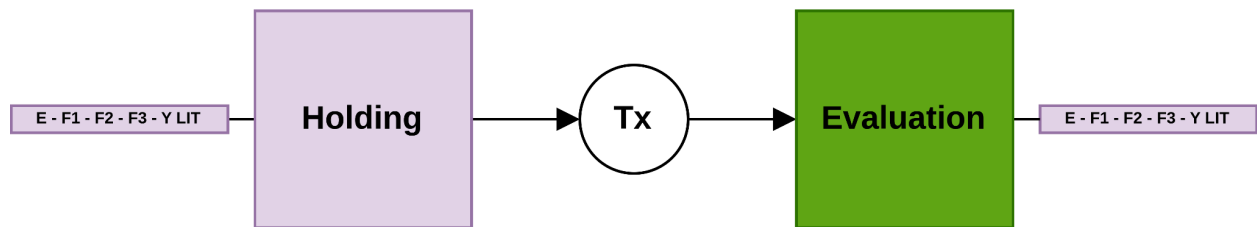
- L81 defines tokensConserved checking that the output holding box maintains the same tokens.
- L82 defines statements checking that all of the following must be true:
 - authenticMiner, onlyOne, validUTXO, rewardConserved, validNISP, treeUpdated, incrementedMiners, samePeriod, and tokensConserved.
- L96 checks that statements is true and signerProp is true.
 - Why not define CTX_SIGNER as a SigmaProp and use that in the above conditions instead of also having signerProp be executed from the transaction context?
- Otherwise, the NISP submission phase has ended and the following is checked to ensure that the holding box becomes the evaluation box:
 - L100 defines nextBlock getting a Long value from register 8 of the output evaluation box.
 - L101 defines conserved values checking that the output evaluation box has the following register value preserved:
 - The ERG emission value.
 - The tokens.
 - The NISP tree.
 - The miner NISP submission count.
 - The total score.
 - The next period is updated to the current height which must be greater than the submission period start plus the submission period length.
 - The current box id equals the first input box id to prevent duplicate holding boxes in the same transaction.
 - R8 of the evaluation box represented by nextBlock equals the currentPeriod value from the holding box. So the submission period start height is carried over to the evaluation box.
 - L115 defines isEvaluationContract checking that the blake2b256 hash of the evaluation box proposition bytes equals the CONST_EVAL_PROPBYTES compile-time constant.
 - L117 checks that isEvaluationContract and conservedValues are true.

Transaction Diagrams

Holding: NISP Submission



Holding: Convert to Evaluation



4EYES Labs Audit - Lithos: Mining (Part 2)

Review Comments

- Miners do not have to be P2PK addresses, Lithos allows anyone to mine under any script. At the time of this writing, this functionality is not exposed in the Lithos client yet.
- By using the ErgoScript executeFromVar function it enables miners under any script to mine as part of the Lithos pool and receive rewards.

Evaluation

Description

- During the evaluation phase, the evaluation box is used for checking that valid fraud-proofs are used for checking submitted NISPs. However, specific fraud-proof logic is contained in the corresponding fraud-proof contract.
- Once the evaluation phase ends, the evaluation box turns into the payout box.
- The fraud-proof evaluation is optimistic in the sense that it assumes initially that miners provide valid NISPs and an evaluation transaction only occurs if someone finds valid fraud. So, ideally, there should be no evaluation transactions occurring if there is no fraud.

Box Contents

Registers

- R4 [AvlTree]: The NISP tree filled with NISPs submitted by miners during the submission phase.
- R5 [Int]: Total miners in a given period, representing miners who have posted a NISP. This value is decremented after a valid fraud-proof is posted.
- R6 [BigInt]: The total share score from the NISP submission period. The Share Score of a NISP is the first 8 bytes of the NISP. Individual Share Scores have type Long. Share scores are removed after a valid fraud-proof is posted.
- R7 [Long]: Blockheight representing the start of the evaluation period, updated after the initial NISP submission period.
- R8 [Long]: Blockheight representing the start of the NISP submission period, carried over from the submission phase.

Tokens

- Index 0: LIT tokens to be emitted to miners of the pool.

Context Variables

- Index 0 [Coll[Byte]]: The fraud proof script.

Compile-Time Constants

- CONST_PERIOD_LENGTH [Int]: The length of the evaluation period, used for determining the appropriate spending path in the holding contract.
- CONST_PAYOUT_PROPBYTES [Coll[Byte]]: The hashed proposition bytes of the evaluation contract.
- CONST_FP_ID [Coll[Byte]]: Singleton token id for the box holding the valid fraud-proofs.

Contract Analysis

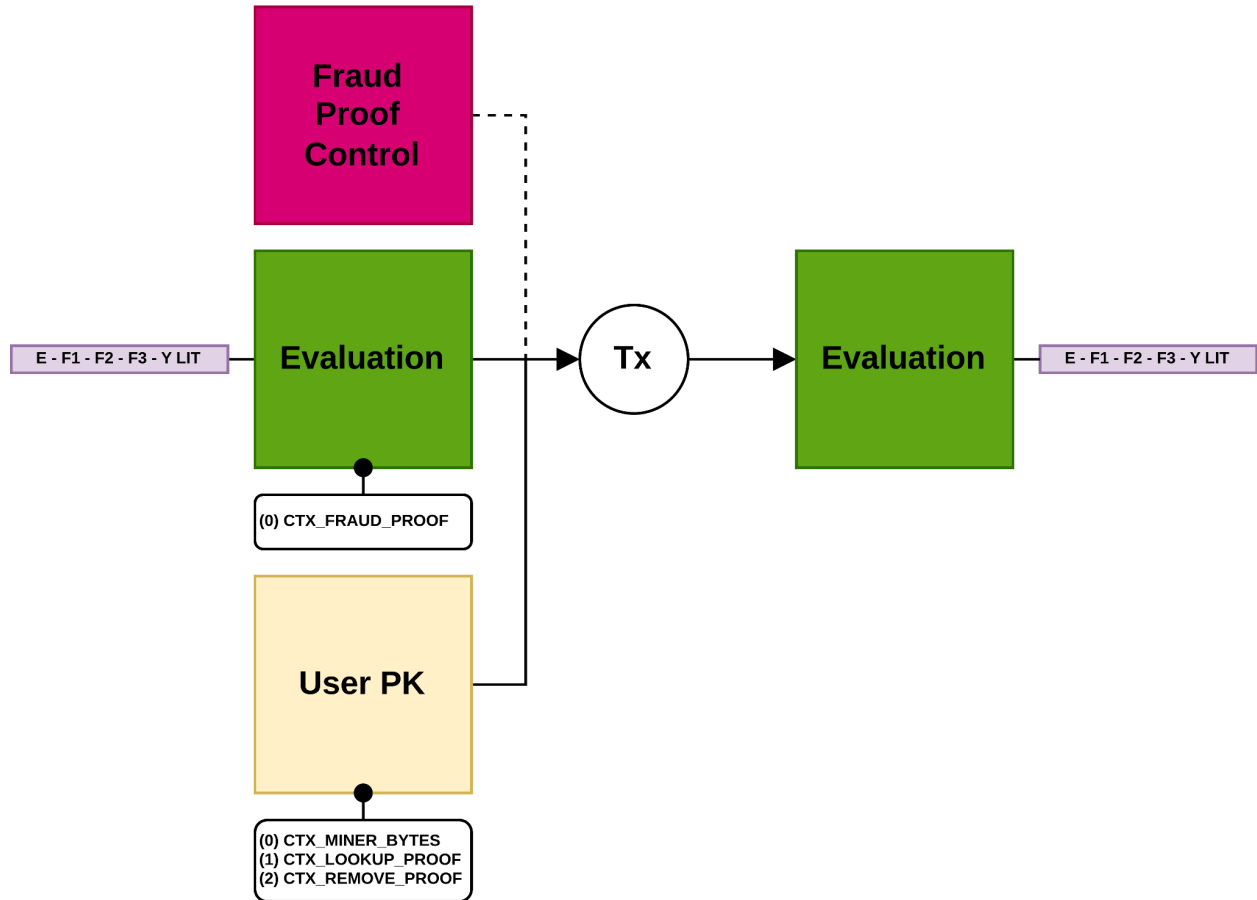
- L47 defines onlyOne checking that there is only one evaluation box in the inputs.
- L52 checks that if the height of the current block being validated is within the evaluation phase then the following is checked:
 - L53 defines fpData as the first data-input in the transaction. This box contains the hashed proposition bytes of the valid fraud-proof contracts that can be spent with the evaluation box.
 - L54 defines authenticFPData checking that the data-input contains the CONST_FP_ID singleton token at index 0.
 - L55 defines script getting the fraud-proof script bytes as a context-variable at index 0.
 - L58 defines fpSet as the collection of valid hashed fraud-proof proposition-bytes from R4 of the fpData data-input.
 - L59 defines validFP checking that there exists a fraud-proof hash in fpSet that matches the blake2b256 hash of the fraud-proof script context-variable.
 - L63 defines contractSame checking that the output evaluation box has the same contract as the input evaluation box.
 - L64 defines correctInputs checking that there are only two inputs in the transaction.
 - L65 defines tokensConserved checking that the output evaluation box contains the same tokens as the input evaluation box.
 - L66 defines flagsConserved checking all of the following properties of the AVL tree in the output evaluation box are true:
 - Insertions, updates, and deletions are enabled.
 - The key length is the same as the input AVL tree key length.
 - The value length option is the same as the input AVL tree value length option.
 - This is set to be empty in the Collateral_Mainnet.ergo contract, thus values inserted into the tree can be of different lengths.
 - These options are set in the Collateral_Mainnet.ergo contract so why do they need to be checked again?
 - L75 checks that validFP, authenticFPData, onlyOne, contractSame, correctInputs, tokensConserved, flagsConserved, and the fraud-proof contract evaluate to true.
- Otherwise, the evaluation phase is over and the evaluation box is converted into the payout box, where the following is checked:
 - L84 defines passedEvalPeriod checking that the height of the current block being validated is greater than or equal to the blockheight corresponding to the end of the evaluation phase.
 - L85 defines tokensAdded checking that:
 - If the evaluation box contains at least 1 token, then the output payout box has in R8 a Long value corresponding to the amount of the token at index 0.
 - Otherwise, return true.

4EYES Labs Audit - Lithos: Mining (Part 2)

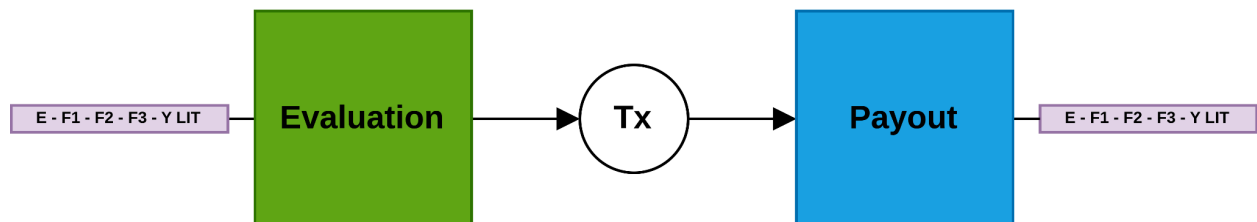
- This stores the LIT block reward value in R8 of the payout box.
- L91 defines conservedValues checking that the following properties of the output payout box are the same as the input evaluation box:
 - The ERG value, the tokens, the NISP AVL tree, the miner count, and the total share score.
 - The period becomes the ERG block reward value (this just means that the register value that used to be the period is now set to the Long value corresponding to the ERG value in the current box).
 - tokensAdded is checked so that if there is a token, then the amount gets added to R8 of the payout box.
- L103 defines isPayoutContract checking that the blake2b256 hash of the payout box equals the CONST_PAYOUT_PROPBYTES compile-time constant.
- L105 checks that isPayoutContract, conservedValues, passedEvalPeriod, and onlyOne are true.

Transaction Diagrams

Evaluation: Fraud-Proof Validation



Evaluation: Convert to Payout



4EYES Labs Audit - Lithos: Mining (Part 2)

Review Comments

- Fraud proofs check the AVL tree digest only, so the Evaluation contract enforces the remaining flags. If a spending path enforces the digest flags for the tree in the output box, then in another spending path that carries the tree, the flags could be changed if not checked again.

Payout

Description

- Contains the ERG and LIT to be paid-out to miners of the Lithos decentralized mining pool.
- If no miners are in the NISP tree, then anyone can spend the box as long as there is no miner fee output box.
- If there is any remaining ERG in the input payout box, then it will be distributed along with any remaining LIT tokens in a subsequent transaction.

Box Contents

Registers

- R4 [AvlTree]: The NISP tree filled with NISPs submitted by miners during the submission phase.
- R5 [Int]: Total miners in a given period, representing miners who have posted a NISP. This value is decremented after a valid fraud-proof is posted. Once in the payout box, this value is constant.
- R6 [BigInt]: The total share score from the NISP submission period. The Share Score of a NISP is the first 8 bytes of the NISP. Individual Share Scores have type Long. Share scores are removed after a valid fraud-proof is posted. Once in the payout box, this value is constant.
- R7 [Long]: ERG value in the current box.
- R8 [Long]: The amount of LIT tokens to be distributed as rewards to miners.

Tokens

- Index 0: LIT tokens to be emitted to miners of the pool.

Context Variables

- Index 0 [Coll[Coll[Byte]]]: Data representing the hashed proposition bytes of miners to retrieve from the NISP AVL Tree.
- Index 1 [Coll[Byte]]: The lookup proof for the AVL tree.
- Index 2 [CTX_REMOVE_PROOF]: The remove proof for the AVL tree.

Compile-Time Constants

- CONST_FEE_HASH [Coll[Byte]]: The hashed proposition bytes of the fee contract.

Contract Analysis

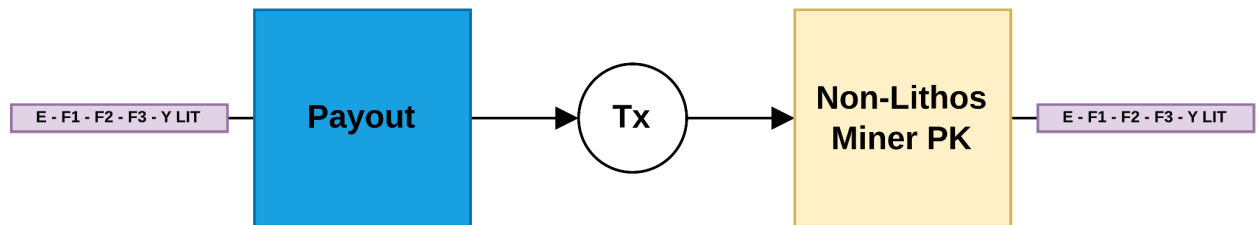
- If the amount of miners is 0, either no miners submitted NISP proofs or all were removed from the AVL tree due to fraud, then the following is checked:
 - L42 defines hasFeeOutput checking that there exists an output box whose proposition bytes hash equals the CONST_FEE_HASH compile-time constant.
 - L46 defines onlyOne checking that there is only one payout box in the inputs.
 - L49 defines noFeeOutput setting hasFeeOutput to its negation.
 - L50 checks that noFeeOutput and onlyOne are true.
 - This spending path ensures that the transaction has no miner fee output, thus rational miners will not include this transaction in their block candidate, so only the block producer will include it, setting the output to their own address (since there is no restriction on where the ERG can go, other than the normal miner fee box).
- Otherwise, miners of the pool who were not removed from the NISP AVL tree due to fraud get rewarded accordingly:
 - L55 defines minersToPay as a collection of tuples containing the miner hashed proposition bytes and their NISP.
 - L59 defines minersWithScore as the collection minersToPay where only the first 8 bytes from the NISP proof are converted to a Long representing the share score.
 - L66 defines minersWithReward as the collection minersWithScore where the share score is replaced with the proportional amount of ERG rewards based on the share score.
 - L72 defines minersWithTokens as the collection minersWithScore where the share score is replaced with the proportional amount of LIT rewards based on the share score.
 - L78 defines totalMinerRewards as the sum of the calculated proportional miner ERG rewards from minersWithReward.
 - L83 defines totalTokenRewards as the sum of the calculated proportional miner LIT reward from minersWithTokens.
 - If the current box value minus the total ERG rewards calculated is greater than 1_000_000L nanoERG, then the following is checked:
 - L92-L98 defines the register values from the output payout box.
 - L99 defines the slice of the outputs after the payout box representing the miners getting paid.
 - L102 defines treeUpdated checking that miners in CTX_KEY_DATA are removed from the NISP tree and that this new tree is in the output payout box.
 - L106 defines minersRewarded checking that all miners in the output slice minerOutputBoxes have the corresponding proposition bytes and value from the minersWithReward collection.
 - L110 defines tokensRewarded checking that:
 - If the LIT token reward is greater than 0, then:

4EYES Labs Audit - Lithos: Mining (Part 2)

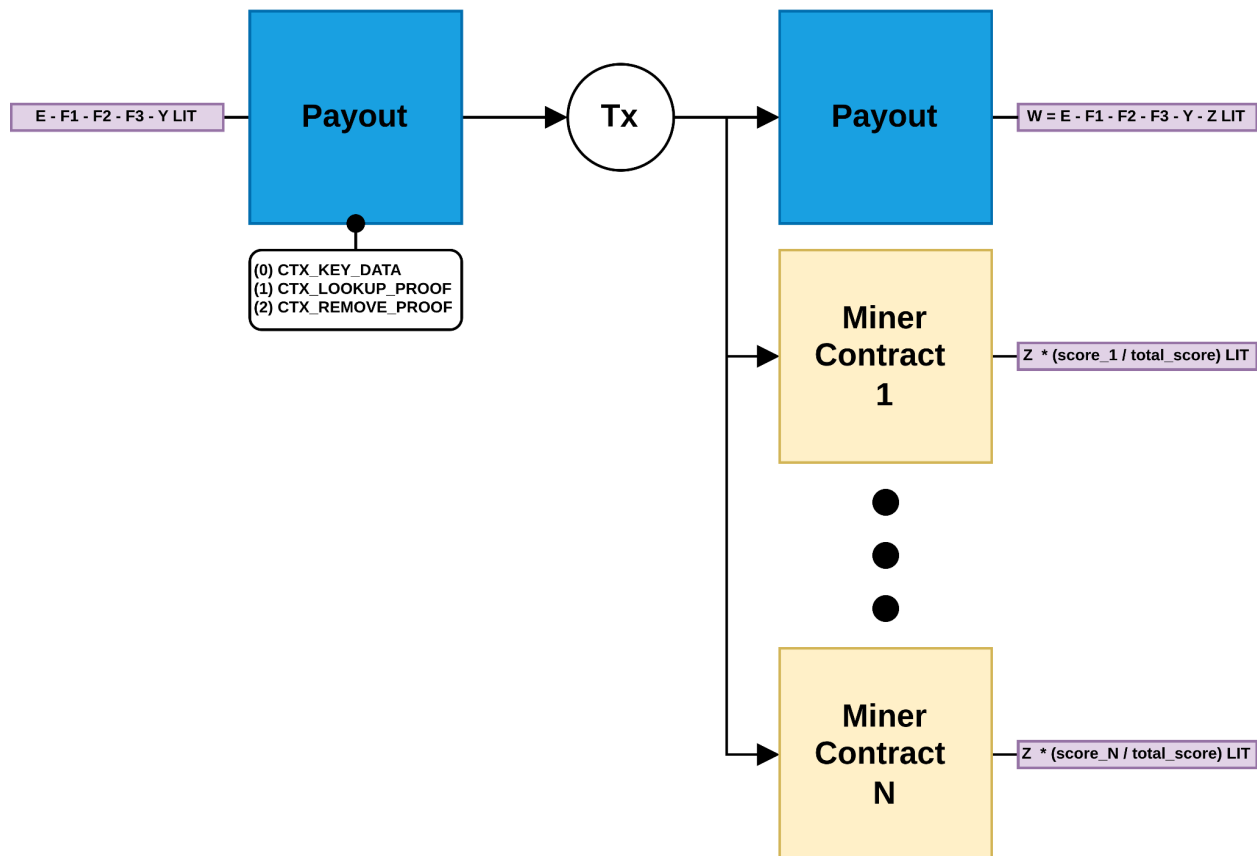
- For all miners in the output slice minerOutputBoxes, check that if the calculated miner token reward is positive then check that the correct token id and token amount is provided to the miner.
 - Otherwise, return true.
 - Otherwise, return true.
- L125 defines validUTXO checking that the output payout box has the same proposition bytes as the input payout box.
- L126 defines usedReward checking that the output payout box value contains the input payout box value minus the calculated totalMinerRewards.
 - Thus, since in this spending path, there could be leftover ERG, it will allow for a subsequent payout transaction where all remaining ERG will be used.
- L127 defines usedTokens checking that the output payout box contains the LIT token id and the amount in the input payout box minus the calculated totalTokenRewards.
- L128-L131 checks that the output payout box keeps the same total reward, total score, miner count, and token reward.
 - Why would we not need to update nextReward with SELF.value - totalMinerRewards?
- L132 defines onlyOne checking that there is only one payout input box.
- L133 defines statements checking that all of the following are true: validUTXO, usedReward, usedTokens, sameScore, sameTotal, sameMiners, sameTokenReward, treeUpdated, minersRewarded, tokensRewarded, onlyOne.
- L148 checks that statements evaluates to true.
- Otherwise, all ERG in the payout box is used to reward miners:
 - L150 defines minerOutputBoxes as the slice of the outputs representing the miners getting paid. There is no payout output box.
 - L153 defines minersRewarded checking that all miners in the collection minerWithReward have the required ERG and that they exist in the output slice minerOutputBoxes.
 - L158 defines tokensRewarded in the same way as in the previous spending path.
 - L173 defines onlyOne checking that there is only one payout input box.
 - L178 defines statements checking that the following are true: onlyOne, minersRewarded, tokensRewarded.
 - Where does the extra ERG come from for the minimum ERG in the miner outputs?
 - L185 checks that statements evaluates to true.

Transaction Diagrams

Payout: No NISP Miners

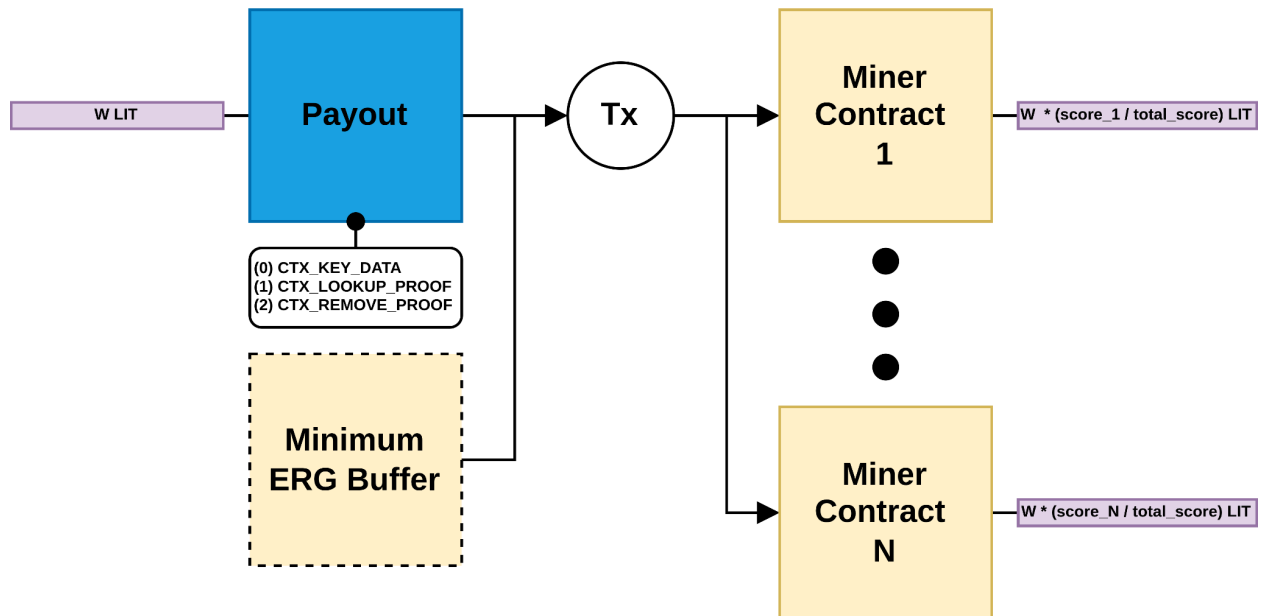


Payout: Leftover ERG



4EYES Labs Audit - Lithos: Mining (Part 2)

Payout: No Leftover ERG



4EYES Labs Audit - Lithos: Mining (Part 2)

Review Comments

- During payout, not all miners in the tree are paid in one transaction. The totalReward is kept so the remaining miners can get paid their share based on the same total. Nonetheless, it is logically possible for some funds to get locked in the payout. However, practically, this is impossible, because it would require 1 million NISP submissions in the holding period, which is only 360 blocks.
- If the miner reward is less than 0.001 ERG then an ERG buffer input would be required. A revised payout contract alleviates this problem.

LithosDex: Liquidity Pool

Description

- The LithosDex liquidity pool contract adapted from the ErgoDex N2T contract.
- The pool uses “virtual” LP tokens emitted during a deposit and absorbed during a redemption.
- The pool enables deposits, redeems, swaps, a provision resize, and flushing the pending fees to the fee vault.
- A liquidity provision is recorded in a provision box and emits an NFT representing this provision.

Box Contents

Registers

- R4 [Long]: LP tokens liquidity, the supply is capped by the `CONST_LOCKED_LP` constant.
- R5 [Coll[Long]]: Fee parameters consisting of the following: the dex fee, the ERG fee, and the token fee.
- R6 [Coll[Long]]: Pending fees to be added to the fee vault contract. The collection has the following order: pendingX (ERG fees), pendingY (token fees). Held separate from their corresponding reserves during calculations.
- R7 [BigInt]: Cumulative ERG fee per share.
- R8 [BigInt]: Cumulative token fee per share.

Tokens

- Index 0: Pool NFT uniquely identifying the liquidity pool for the particular ERG-2-Token pair.
- Index 1: TokenY being traded against ERG.
- Index 1: Provision tokens, one is emitted after a deposit and returned after a redemption.

Context Variables

- Index 0 [Byte]: The `CTX_OP` used for choosing the spending path to take in the contract.
 - 0: Swap - ERG for token or token for ERG.
 - 1: Deposit - Adding liquidity, which mints the ownership NFT and releases a provision token.
 - 2: Redeem - Remove a whole liquidity provision and return the provision token.
 - 3: Flush - Move the accrued fees into the fee vault.
 - 4: Resize - Change a liquidity provision size, either increase or decrease, without closing it.

4EYES Labs Audit - Lithos: Mining (Part 2)

Compile-Time Constants

- `CONST_PROVISION_GUARD_HASH [Coll[Byte]]`: The hashed proposition bytes of the provision guard contract.
- `CONST_VAULT_NFT [Coll[Byte]]`: Singleton NFT uniquely identifying the fee vault contract.

Contract Analysis

- L147 defines `reservesHeld` checking that the ERG reserves in the input and output pool boxes remains positive.
- L149-L164 defines `poolPreserved` checking that the following input and output pool properties:
 - There is only one pool box in the input.
 - The pools have the same contract.
 - The pools have the same NFT.
 - The pools have the same TokenY token id.
 - The pools have the same provision token id.
 - The output pool has three unique tokens.
 - The pools have the same fee parameters.
 - The output pool must have an ERG value greater than the `CONST_MINT_RENT` value of 10_000_000 nanoERG (0.01 ERG).
 - `reservesHeld` is true.
 - The output pool's pending fee collection size remains as 2.
- L168 defines `accHeld` checking that the accumulated ERG and TokenY fees are the same in the input and output pools.
 - Only a Swap transaction should change the accumulators.
- L171 defines `provTokenHeld` checking that the number of provision tokens in the input equals the number of provision tokens in the input pool.
- L173 defines `validAction` checking that:
 - If `CTX_OP` is 0 (Swap) then:
 - L178 defines `accXAdvanced` checking that the cumulative ERG fee in the output pool has increased. The cumulative ERG fee increases by the following factor:
 - The change in the cumulative ERG fee per LP share at swap time divided by the LP circulating supply at swap time, multiplied by a scale factor `CONST_SCALE` to prevent integer division truncation.
 - L180 defines `accYAdvanced` checking that the cumulative TokenY fee in the output pool has increased. The cumulative TokenY fee increases by the following factor:
 - The change in the cumulative TokenY fee per LP share at swap time divided by the LP circulating supply at swap time, multiplied by a scale factor `CONST_SCALE` to prevent integer division truncation.
 - L183 defines common checking that:

4EYES Labs Audit - Lithos: Mining (Part 2)

- The change in the LP token circulating supply is 0, since during a swap no “virtual” LP tokens should be emitted from the pool.
- provTokenHeld is true, since no provision token should be emitted during a swap.
- The provision tokens in the input pool and output pool should be equal.
- The accXAdvanced and accYAdvanced are true since a swap happened, so trading fees must have been accumulated from the swap.
- The net change in the ERG value and TokenY amount of the input and output pool must not equal 0, thus ensuring that a swap happened.
 - totalDeltaX > 0 and totalDeltaY < 0 if ERG added and TokenY removed.
 - totalDeltaX < 0 and totalDelta Y > 0 if TokenY added and ERG removed.
 - Having only totalDeltaX != 0 or totalDeltaY != 0 as the condition could allow for both to be positive or negative, i.e. “donations” ?
- L189 checks that if ERG if the net reserves are positive, thus ERG is being added into the pool for the swap (so TokenY being purchased and removed from the pool) then the following is checked:
 - L190 defines satisfiesAMM checking that the AMM CP formula is valid for depositing ERG and releasing TokenY.
 - L192 defines feeXAdded checking that the change in pending cumulative ERG fee per share equals the total change in ERG of the pool times the constant ERG fee per share parameter.
 - Pending ERG fees per share are a fraction of the total ERG added to the liquidity pool.
 - Dex fee is taken after pending ERG fee.
 - L193 checks that satisfiesAMM is true, feeXAdded is true, that deltaPendingY is 0, and common is true.
 - The pending fee is always taken from the input, so during an ERG-2-Token swap, the pending fee is taken from the input ERG amount.
- Otherwise, there is a Token-2-ERG swap, where the following is checked:
 - L195 defines satisfiesAMM checking that the AMM CP formula is valid for depositing TokenY and releasing ERG.
 - L192 defines feeYAdded checking that the change in pending cumulative TokenY fee per share equals the total change in TokenY of the pool times the constant token fee per share parameter.
 - Pending TokenY fees per share are a fraction of the total TokenY added to the liquidity pool.

4EYES Labs Audit - Lithos: Mining (Part 2)

- Dex fee is taken after pending TokenY fee.
 - L193 checks that satisfiesAMM is true, feeYAdded is true, that deltaPendingX is 0, and common is true.
 - The pending fee is always taken from the input, so during an Token-2-ERG swap, the pending fee is taken from the input TokenY amount.
- If CTX_OP is 1 (Deposit) then:
 - L206 defines sharesUnlocked as the minimum of the following values:
 - The total change in ERG reserves multiplied by the input LP token circulating supply, divided by the input reserves.
 - The total change in TokenY reserves multiplied by the input LP token circulating supply, divided by the input reserves.
 - Thus, a provision gets an amount of virtual LP tokens equivalent to a fraction of the current LP token circulating supply, where this fraction is determined by the minimum of the relative amount ERG or TokenY from the pair added in the provision.
 - This defines an upper-bound on the amount of virtual LP tokens emitted.
 - L211-L237 checks that all of the following are true:
 - accHeld is true, the fee accumulators in the output pool equals the fee accumulators in the input pool.
 - provTokenHeld is true, so the amount of provision tokens in the inputs equals the amount of provision tokens in the input pool.
 - deltaPendingX and deltaPendingY equals 0, so there are no pending fees added in the output pool.
 - deltaSupplyLP is greater than 0 since during a provision, virtual LP tokens are emitted, thus the circulating supply increases.
 - deltaSupplyLP must be less than or equal to sharesUnlocked.
 - The hash of the newly created output provision box proposition bytes equals the expected CONST_PROVISION_GUARD_HASH compile-time constant.
 - The output provision box has exactly one provision token whose token id equals the provision token id of the liquidity pool.
 - The ERG value of the provision box is greater than or equal to CONST_PROVISION_MIN.
 - The provision box R8 with type Coll[Byte] is not defined.
 - Since a provision box represents a liquidity position, it is possible for the provision box to remain unspent for many years.
 - This condition forces the box to have a fixed set of registers to minimize its byte size and thus the storage rent amount.

4EYES Labs Audit - Lithos: Mining (Part 2)

- The amount of ERG the provision box is initialized with, while also accounting for the total box size, can withstand two storage rent rounds, so ~8 years as of now.
- The provision box R4 has type BigInt and is set to the input ERG accX value.
- The provision box R5 has type BigInt and is set to the input TokenY accY value.
- The provision box R6 has type Coll[Byte] and is set to the box id of the input pool, this represents a virtual NFT minted.
 - All provision boxes contain only one token equal to the provision token, but each is uniquely identified by the id in R6.
- The provision box R7 has type Long and is set to deltaSupplyLP, the change in supply of LP tokens from the input and output pool boxes. This represents the share of virtual LP tokens the provision gets.
- The number of tokens in the outputs with token id equal to the box id of the input pool box should be 1.
 - This minted NFT represents the liquidity provision. The box it corresponds to is the provision box with the id in R6.
- The amount of provision tokens in the output pool should decrease by one.
- If CTX_OP is 2 (Redeem) then:
 - L246 defines satisfiesAMM checking that the AMM redeem equation is satisfied for both ERG addition and TokenY being removed from the pool in an amount at most defined by the change in LP circulating supply.
 - The change in LP circulating supply is determined by the LP share of the provision.
 - L250-L265 checks the following:
 - accHeld is true, the fee accumulators in the output pool equals the fee accumulators in the input pool.
 - deltaPendingX and deltaPendingY equals 0, so there are no pending fees added in the output pool.
 - The hash of the provision input box proposition bytes equals the expected CONST_PROVISION_GUARD_HASH compile-time constant.
 - The provision box contains exactly one provision token.
 - satisfiesAMM is true.
 - The circulating supply of LP tokens in the output pool box is greater than or equal to the CONST_MIN_SUPPLY value.
 - Since deltaSupplyLP will be negative, -1 times the value must equal the provision amount corresponding to the share of LP tokens in the provision box.

4EYES Labs Audit - Lithos: Mining (Part 2)

- The number of provision tokens in the inputs must equal the number of provision tokens in the input pool plus 1.
 - The provision token amount in the output pool must equal the number of provision tokens in the input pool plus 1.
- If CTX_OP is 3 (Flush) then all of the following must be true:
 - accHeld, since pending fees are being added to the vault, the fee accumulators should not change.
 - The number of provision tokens in the inputs should equal the number of provision tokens in the input liquidity pool.
 - The provision token id in the output pool should equal the provision token id in the input pool.
 - The number of pending ERG fees or TokenY fees in the input pool must be positive, i.e. there must be pending fees to move into the vault for a valid flush transaction.
 - The input vault box must have the CONST_VAULT_NFT as an index 0 token.
 - The output vault must have the same NFT at index 0 as the input vault.
 - The total change in ERG from the output and input pools must equal the negative of the pending ERG fees in the input pool, i.e. all of the pending ERG fees must move out of the input pool into the vault.
 - The total change in TokenY from the output and input pools must equal the negative of the pending TokenY fees in the input pool, i.e. all of the pending TokenY fees must move out of the input pool into the vault.
 - The output vault ERG value must be greater than or equal to the input vault ERG value plus the pending ERG fees from the input pool.
 - This allows a transaction to add additional ERG into the vault from outside of the input liquidity pool.
 - The output vault TokenY value must be greater than or equal to the input vault TokenY value plus the pending TokenY fees from the input pool.
 - This allows a transaction to add additional TokenY into the vault from outside of the input liquidity pool.
 - The output vault TokenY id must equal the TokenY id of the input pool or the output vault TokenY amount must equal 0.
 - Due to the way the smartcontract is written, this ensures that the checks are still valid if only ERG fees are being transferred to the vault.
 - R4 of the output fee vault is a BigInt equal to the ERG fee accumulator of the input pool.
 - R5 of the output fee vault is a BigInt equal to the TokenY fee accumulator of the input pool.
 - The pending ERG fees and TokenY fees of the output pool are set to 0.
 - The change in LP circulating supply should be 0 since no virtual LP tokens are being added or released from the liquidity pool.
 - The change in ERG reserves and TokenY reserves should be 0.

4EYES Labs Audit - Lithos: Mining (Part 2)

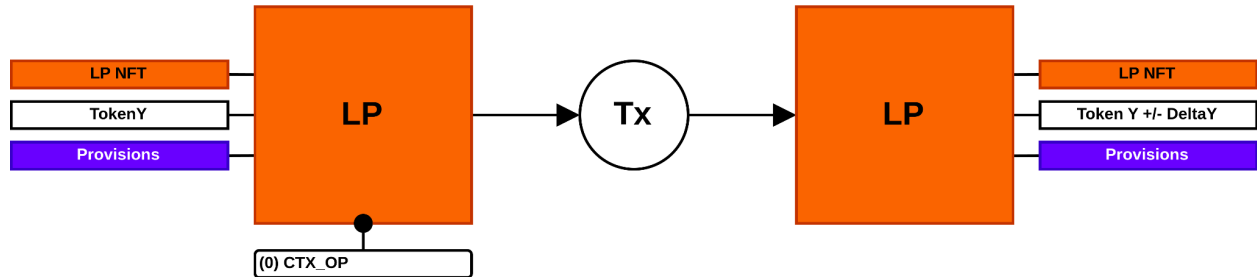
- If CTX_OP is 4 (Resize) then:
 - L315 defines resizeAMM checking that:
 - If the change in LP token circulating supply from the output and input pool is greater than 0, thus the provision increased in size, then:
 - L317 defines sharesUnlocked the same as at L206.
 - L321 checks that deltaSupplyLP is less than or equal to sharesUnlocked.
 - Otherwise, deltaSupplyLP is negative, thus the provision decreased in size, then:
 - The AMM equation for redeeming is checked the same as at L246.
 - L329-L381 checks that all of the following are true:
 - accHeld, since pending fees are being added to the vault, the fee accumulators should not change.
 - resizeAMM is true, so either a deposit for increasing the provision size or a redeem for decreasing the provision size.
 - The circulating supply of LP tokens in the output pool box is greater than or equal to the CONST_MIN_SUPPLY value.
 - The change in LP token circulating supply must equal the difference between the LP provision recorded in the output and input provision boxes.
 - The change in LP token circulating supply cannot be 0.
 - The LP provision recorded in the output provision box must be positive.
 - Unlike a redeem, resize does not return the provision token or burn the provision NFT.
 - The hash of the provision input box proposition bytes equals the expected CONST_PROVISION_GUARD_HASH compile-time constant.
 - The provision input box contains exactly one provision token.
 - The hash of the provision output box proposition bytes equals the expected CONST_PROVISION_GUARD_HASH compile-time constant.
 - The provision output box contains exactly one provision token.
 - The provision output box contains exactly one unique token.
 - The ERG value of the provision output box is greater than or equal to CONST_PROVISION_MIN.
 - The provision output box R8 with type Coll[Byte] is not defined.
 - The provision input and output box R6 referencing the provision NFT are the same.
 - The number of provision tokens in the input equals the number of provision tokens in the input pool plus 1.

4EYES Labs Audit - Lithos: Mining (Part 2)

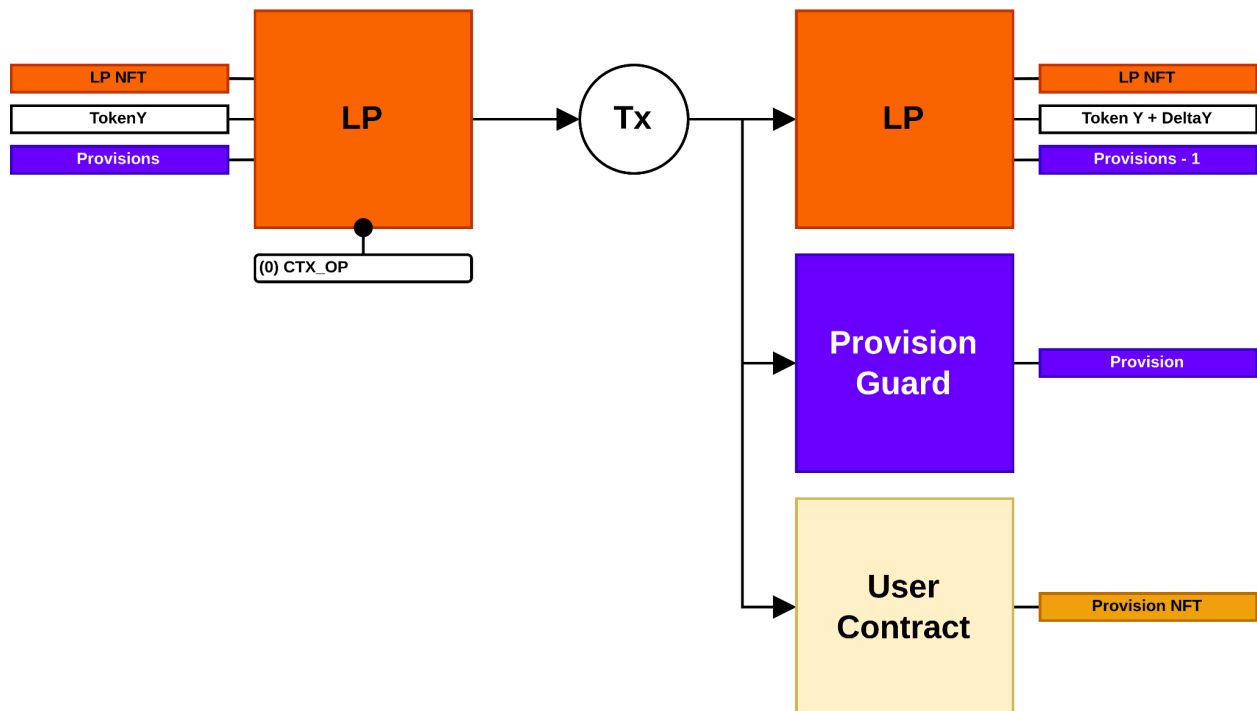
- The number of provision tokens in the output equals the number of provision tokens in the input pool.
- L363-L364 update the new ERG and TokenY fee accumulators in the output provision box. The idea is that the fees owed to the provision are the same right before and right after the resize. Due to integer division, it reduces the claimable fees after the resize.
- L371-L379 flushes the pending ERG and TokenY fees to the vault contract the same as in L281-L294.
- L388 checks that poolPreserved and validAction are true.

Transaction Diagrams

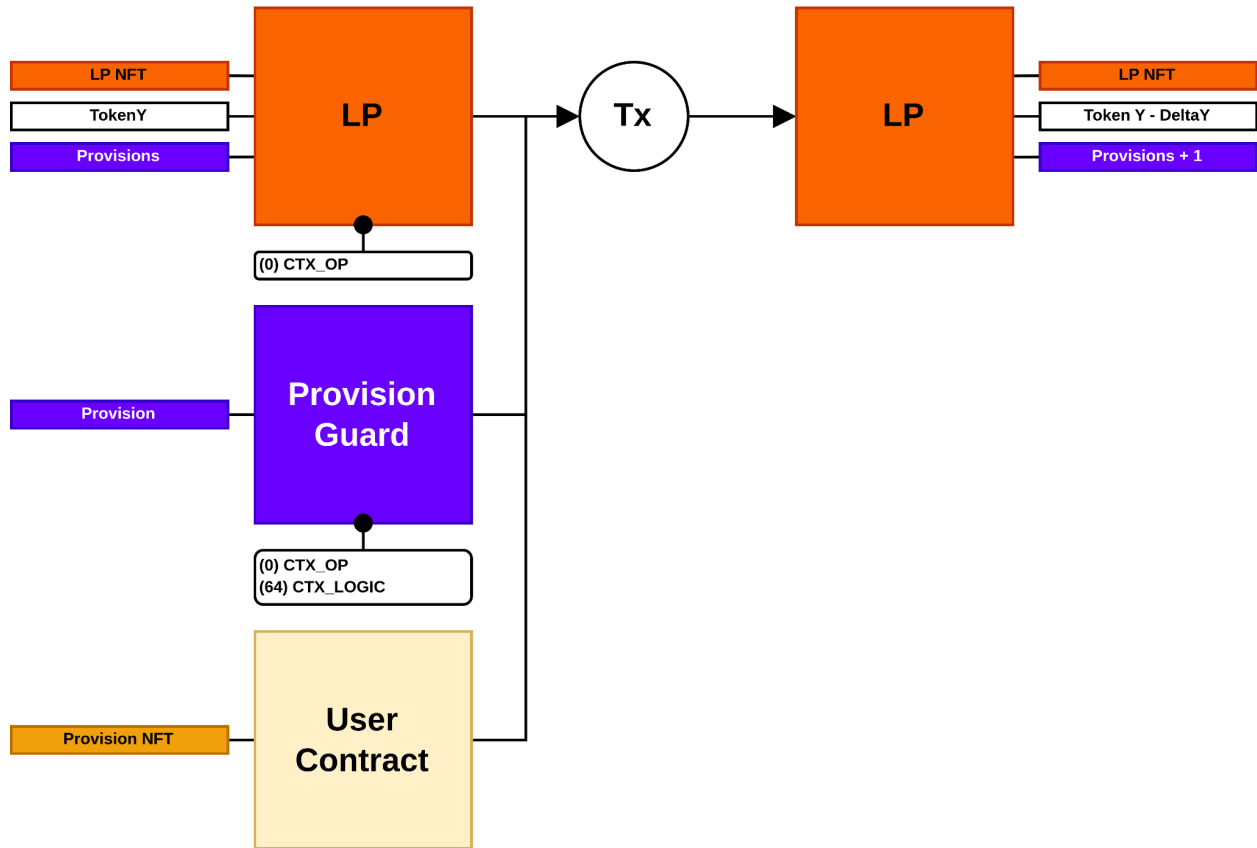
LithosDex: Swap



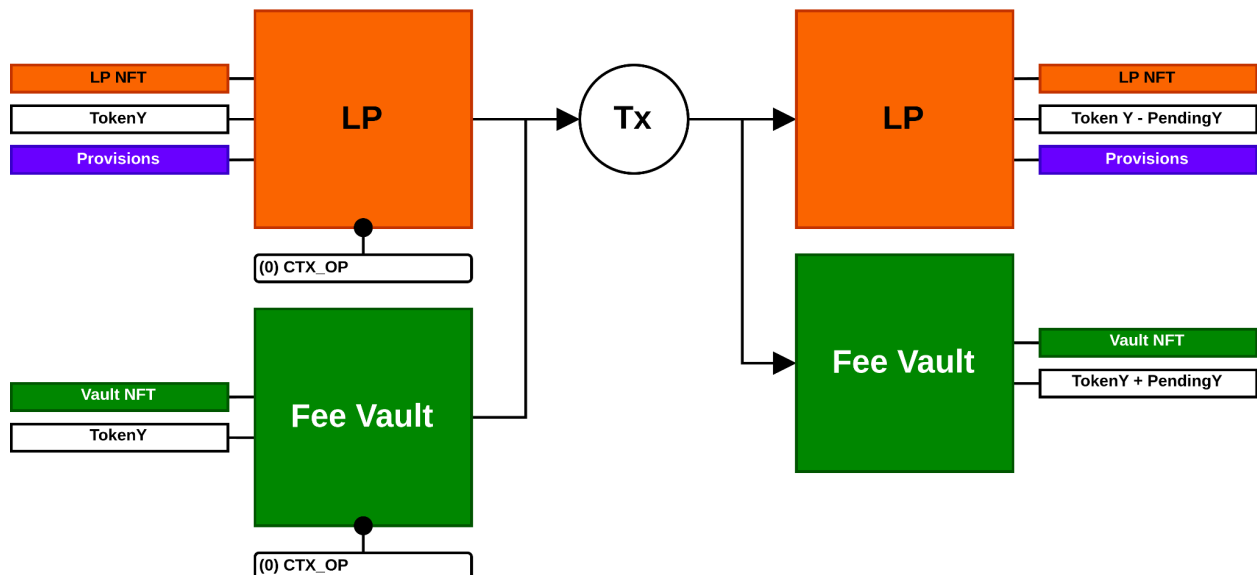
LithosDex: Deposit



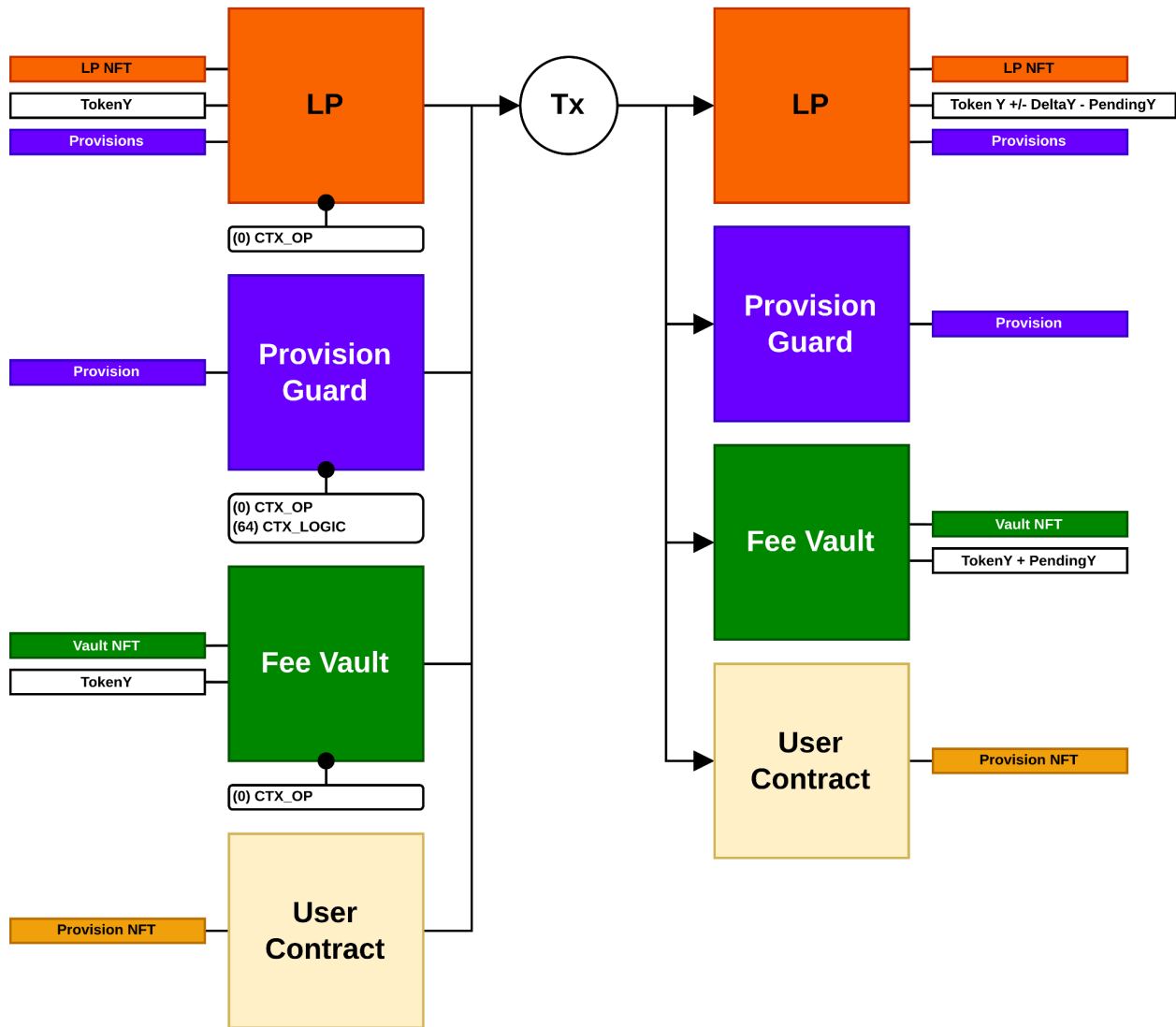
LithosDex: Redeem



LithosDex: Flush



LithosDex: Resize



4EYES Labs Audit - Lithos: Mining (Part 2)

Review Comments

- Yes, donations are technically possible.

LithosDex: Fee Vault

Description

- The liquidity pool fee vault where pending ERG and TokenY fees are added.
- By adding owed fees to a vault box, provision fees can be claimed without spending the liquidity pool box, claiming provision fees does not close the provision.

Box Contents

Registers

- R4 [BigInt]: The liquidity pool ERG fee accumulator from the last flush transaction.
- R5 [BigInt]: The liquidity pool TokenY fee accumulator from the last flush transaction.

Tokens

- Index 0: Fee vault NFT uniquely identifying the box.
- Index 1: TokenY pending fees.

Context Variables

- Index 0 [Byte]: The CTX_OP used for choosing the spending path to take in the contract.
 - 0: Claim - One or more provision boxes can claim their owed fees.
 - 1: Flush - Receive the liquidity pool's pending ERG and TokenY fees.
- Index 1 [Int]: The CTX_COUNT representing the amount of input provision boxes. Input provision boxes are expected to occupy indices 1..CTX_COUNT with matching output index.

Compile-Time Constants

- CONST_POOL_NFT [Coll[Byte]]: Token id of the liquidity pool NFT.
- CONST_PROV_TOKEN [Coll[Byte]]: Token id of the liquidity pool provision tokens.

Contract Analysis

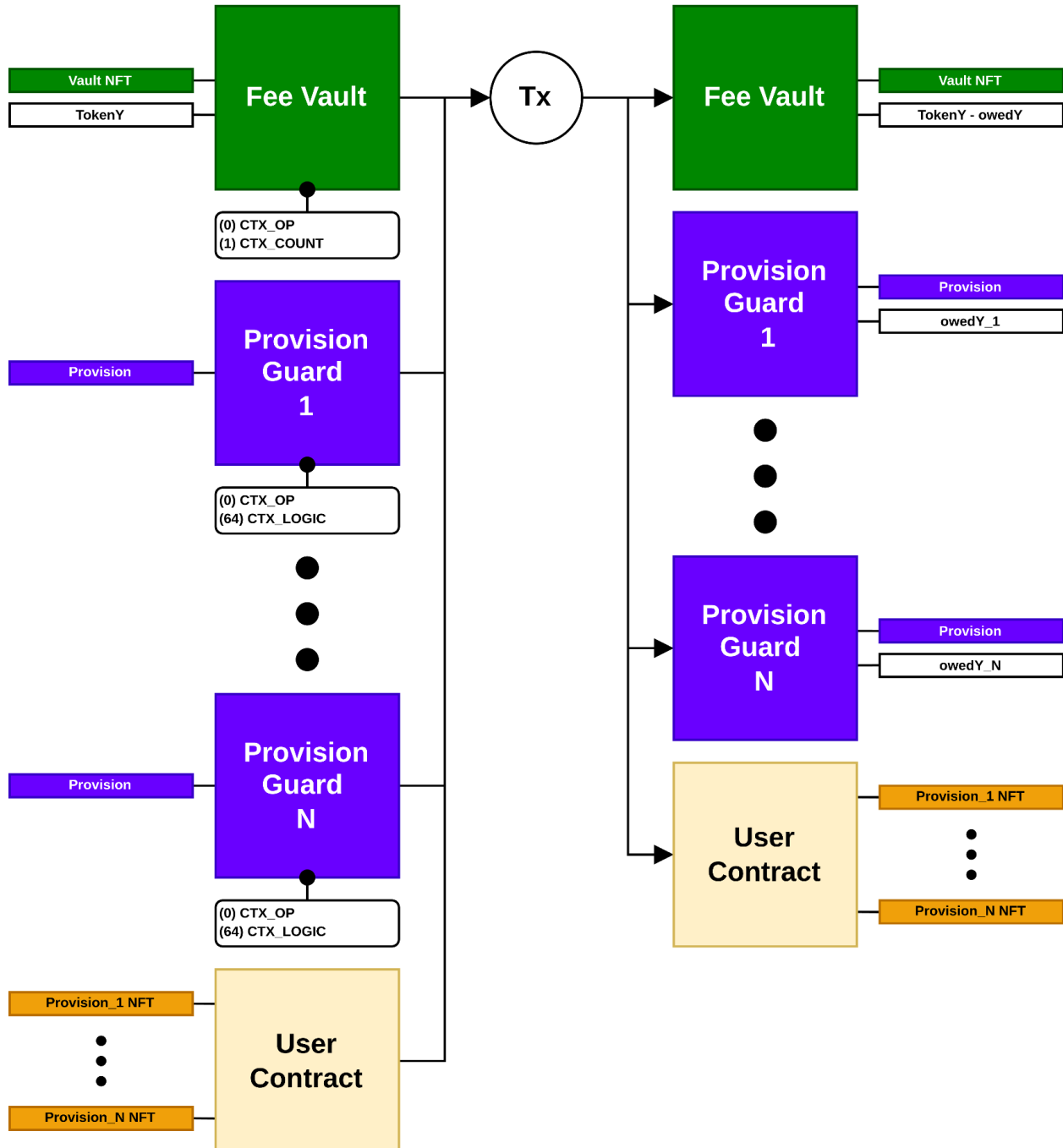
- L45 defines successor expecting the output fee vault to be at output index `CONTEXT.selfBoxIndex`
- L46 defines `onlyOne` checking that there is only one fee vault box at input 0.
- L50-L58 defines `vaultPreserved` checking that all of the following are true:
 - `onlyOne` is true.
 - The output fee vault has the same contract as the input fee vault.
 - The output fee vault has the same vault NFT.
 - The output fee vault has at most 2 unique tokens in it, the other being the accumulated `TokenY` fees.
 - The output fee vault has the same `TokenY` id as the input fee vault or there are no `TokenY` tokens in the output fee vault.
- If `CTX_OP` is 0 (Claim) then the following is checked:
 - L87 defines `settlement` checking the following for each pair of provision input and output:
 - L95-L96 defines `owedX` and `owedY`, calculating the `ERG` and `TokenY` fees, respectively, for the input provision box.
 - When the initial fee accumulator is set for the provision, there is a corresponding `CONST_SCALE` factor implicit in the value. This is removed by the division of the `CONST_SCALE` factor in the current calculation.
 - L98-L112 defines `ok` checking that all of the following are true:
 - The input provision box contains only 1 provision token with token id `CONST_PROV_TOKEN` at index 0.
 - The `ERG` and `TokenY` accumulators in the input provision box are at most the corresponding `ERG` and `TokenY` accumulators of the fee vault.
 - Either the `ERG` or `TokenY` accumulators of the input provision box must be less than the `ERG` or `TokenY` accumulators of the fee vault.
 - If either `ERG` or `TokenY` fee accumulator checks fail then the entire claim fails for all input provision boxes.
 - A provision must claim both `ERG` and `TokenY` fees, not one or the other.
 - The output provision box sets `R4` and `R5` to a `BigInt` representing the `ERG` and `TokenY` accumulators of the fee vault, thus resetting the provision's owed fees.
 - L114 tallies the owed fees and updates the boolean check with the `ok` value.
 - L117 defines `allSettled` checking that the provision boolean check is true and that the `CTX_COUNT` value is greater than 0 ensuring that there is at least one provision box as an input.
 - L120-L121 defines `owedX` and `owedY` representing the summed owed `ERG` and `TokenY` fees from each input provision box.

4EYES Labs Audit - Lithos: Mining (Part 2)

- L123-L131 defines paidOut checking that all of the following are true:
 - The output fee vault ERG value is at least the input fee vault ERG value minus the tallied owedX ERG fees.
 - The output fee vault has at least CONST_VAULT_MIN amount of ERG.
 - The output fee vault TokenY amount is at least the input fee vault TokenY amount minus the tallied owedY TokenY fees.
 - The output fee vault has R4 and R5 set to the same BigInt ERG and TokenY fee accumulator values as the input fee vault.
 - L133 checks that vaultPreserved, allSettled, and paidOut are true.
- If CTX_OP is 1 (Flush) then the following is checked:
 - L140 defines poolBox expecting the liquidity pool to be input 0.
 - L146 gets the context-extension variable of the poolBox at index 0.
 - L148-L153 checks that the following are true:
 - The poolBox contains a token at index 0 with id equal to CONST_POOL_NFT.
 - The poolOp context-extension variable is 3 for a flush or a 4 for a resize (from the pool's perspective).
 - The output fee vault box has the same proposition bytes as the input fee vault box.
 - The output fee vault keeps the vaultNFT at index 0.
- Otherwise, evaluate to false.

Transaction Diagrams

LithosDex: Claim



4EYES Labs Audit - Lithos: Mining (Part 2)

Review Comments

- None

LithosDex: Provision

Description

- Contract representing the user's liquidity provision. It contains the virtual LP tokens and the corresponding fee accumulators for ERG and TokenY.
- The provision can claim accumulated fees from the fee vault, be resized, and closed.

Box Contents

Registers

- R4 [BigInt]: The ERG accumulator from when the provision was first created or after a resize.
- R5 [BigInt]: The TokenY accumulator from when the provision was first created or after a resize.
- R6 [Coll[Byte]]: The token id of the owner NFT minted when the provision was first created, representing the liquidity pool input box id at that time.
- R7 [Long]: The virtual LP tokens for the provision.

Tokens

- Index 0: One provision token provided by the liquidity pool when the provision was created.

Context Variables

- Index 0 [Byte]: The CTX_OP used for choosing the spending path to take in the contract.
 - 0: Claim - One or more provision boxes can claim their owed fees.
 - 1: Redeem - Remove a whole liquidity provision and return the provision token.
 - 2: Refresh - Recreate the box.
 - 3: Resize - Change a liquidity provision size, either increase or decrease, without closing it.

Compile-Time Constants

- CONST_POOL_NFT [Coll[Byte]]: Token id of the liquidity pool NFT.
- CONST_VAULT_NFT [Coll[Byte]]: Token id of the fee vault NFT.

Contract Analysis

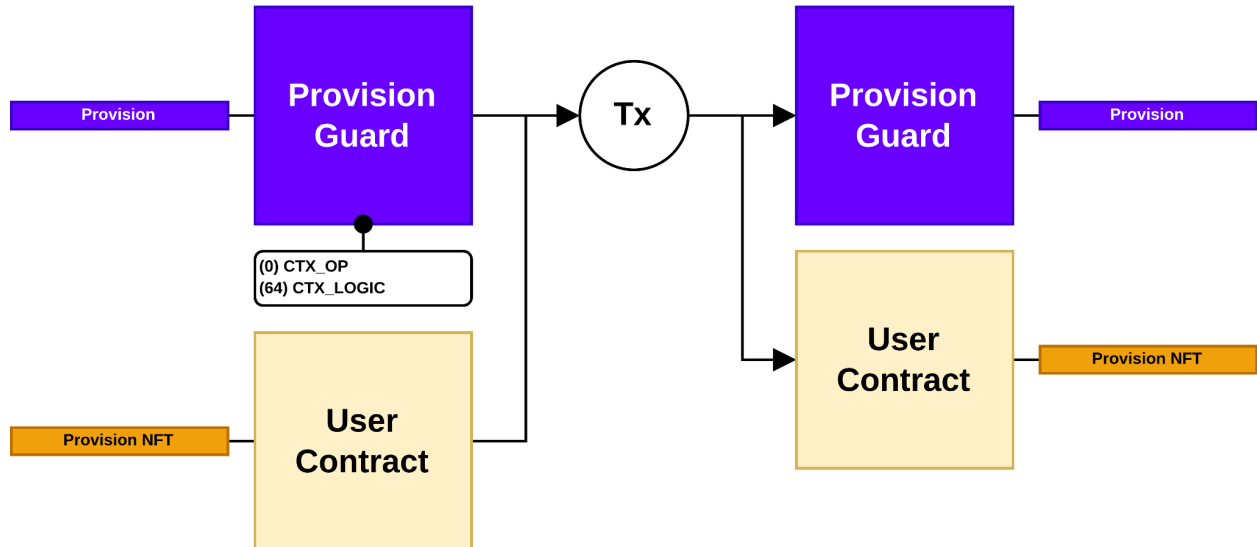
- L49 defines successor as the output at index `CONTEXT.selfBoxIndex`.
- L54-L57 defines `ownerPresent` checking that there is an input containing a token with token id equal to the `ownerNFT`.
- If `CTX_OP` is 0 (Claim) then the following is checked:
 - L76 defines `vaultBox` as input 0.
 - L79 defines `noExtraRegisters` checking that the output provision box does not have R8 defined.
 - This ensures the box maintains the correct byte size to minimize the storage-rent claim.
 - L80-L95 defines `identityHeld` checking that all of the following about the output provision box are true:
 - The proposition bytes are the same.
 - The value increased over the input.
 - The tokens remain the same.
 - The `ownerNFT Coll[Byte]` in R6 does not change.
 - The provision amount Long in R7 does not change.
 - The BigInt fee accumulators for ERG and TokenY increase over the input values in R4 and R4, respectively.
 - `noExtraRegisters` is true.
 - L97 checks that `vaultBox` has the `CONST_VAULT_NFT` at token index 0, that `ownerPresent` is true, and `identityHeld` is true.
 - Why should a claim require the `ownerNFT`?
 - Why do we not check for an output to contain the `ownerNFT`? Since the contracts (fee vault + provision) does not check this, the token could get burned by accident or moved.
- If `CTX_OP` is 1 (Redeem) then the following is checked:
 - L104 defines `authenticPool` checking that input 0 contains the `CONST_POOL_NFT` at token index 0.
 - L106 defines `nftBurned` checking that for all outputs none of them contain the `ownerNFT` token.
 - L111 checks that `ownerPresent`, `authenticPool`, and `nftBurned` are true.
- If `CTX_OP` is 2 (Refresh) then the following is checked:
 - L119 defines `oldEnough` checking that the creation height of the input provision box is less than the current block height minus the `CONST_REFRESH_AGE` value. This checks the box has a minimum age before recreation.
 - L79 defines `noExtraRegisters` checking that the output provision box does not have R8 defined.
 - L124-L138 defines `unchanged` checking that the output provision box has the following:
 - The proposition bytes are the same.
 - The output value is greater than or equal to the input value.
 - The tokens do not change.

4EYES Labs Audit - Lithos: Mining (Part 2)

- The BigInt ERG and TokenY fee accumulator BigInt in R4 and R5, respectively, do not change.
 - The Coll[Byte] ownerNFT in R6 does not change.
 - The provision amount Long in R7 does not change.
 - The output provision box must have an age of at most `CONST_HEIGHT_SLACK`.
- L140 checks that `oldEnough` and `unchanged` are true.
- If `CTX_OP` is 3 (Resize) then the following is checked:
 - L104 defines `authenticPool` checking that input 0 contains the `CONST_POOL_NFT` at token index 0.
 - L151 defines `noExtraRegisters` checking that the output provision box does not have R8 defined.
 - L152 defines `identityHeld` checking the following about the output provision box:
 - The proposition bytes do not change.
 - The output value is greater than or equal to the input value.
 - The tokens do not change.
 - The Coll[Byte] ownerNFT in R6 does not change.
 - `noExtraRegisters` is true.
 - L162 checks that `ownerPresent`, `authenticPool`, and `identityHeld` are true.
 - Why do we not check for an output to contain the ownerNFT? Since the contracts (fee vault + provision) does not check this, the token could get burned by accident or moved.
- Otherwise, evaluate to false.

Transaction Diagrams

LithosDex: Refresh



4EYES Labs Audit - Lithos: Mining (Part 2)

Review Comments

- A provision claim allows for multiple provision input boxes because one user can own multiple NFTs each corresponding to a different provision.
- NFT owners are disincentivized to not burn their NFT during a claim or resize so it was decided that it is unnecessary to check. Thus, anyone interacting with LithosDex outside the official Lithos client (since LithosDex interaction is not restricted) must ensure that the claim and resize transaction are built properly (locally or from a third-party).

LithosDex: Provision Guard

Description

- Contract in the provision box that executes the provision contract via a context-extension variable.

Box Contents

Registers

- Same as the provision contract.

Tokens

- Same as the provision contract.

Context Variables

- Index 64 [Coll[Byte]]: The CTX_LOGIC representing the provision contract serialized bytes..

Compile-Time Constants

- CONST_PROVISION_LOGIC_HASH [Coll[Byte]]: The blake2b256 of the serialized provision contract.

4EYES Labs Audit - Lithos: Mining (Part 2)

Contract Analysis

- L37 defines CTX_LOGIC getting the context-extension variable from index 64.
- L39 checks that:
 - The blake2b256 of CTX_LOGIC equals the
CONST_PROVISION_LOGIC_HASH compile-time constant.
 - The context-extension-variable at index 64 evaluates to a SigmaProp.

Transaction Diagrams

- None

Review Comments

- None

Appendix

Issue Guide

Severity	Description
CRITICAL	The contract contains one or more serious errors and warnings.
SERIOUS	An error that will cause unexpected behavior.
WARNING	A property that is questionable but will not cause unexpected behavior.
GOOD	Everything works as expected.

Status	Description
RESOLVED	The issues have been addressed. Default status if severity is GOOD.
UNRESOLVED	The issues have not been addressed.

About Us

4EYES Labs is an ErgoScript design and auditing company for Ergo blockchain.

Linktree: <https://linktr.ee/4EYESLabs>